

KaminoDatasetGenerator

Relatorio de Estado Atual e Plano de Execucao

Extensao do Pipeline Kamino para C#

Estado atual estimado: **~20-25% concluido**

Data da analise: 10/08/2026

Ultimo commit no repositario: 185f3773 2026-06-25 preparing for AE track

Arquivos nao commitados no momento da analise: 8

2. Resumo Executivo

O objetivo do projeto é estender o pipeline Kamino -- que hoje gera clones Type-IV (semanticamente equivalentes, sintaticamente diferentes) automaticamente a partir do BigCodeBench, mas somente em Python -- para também suportar C#. O plano tem seis etapas: estudar o pipeline existente, mapear diferenças Python/C#, prototipar a tradução Python-C# (5-10 funções), montar um Golden Dataset C# de 100 entradas, gerar clones Type-IV em C# (~7 por entrada, ~700 no total) e avaliar tudo com a infraestrutura de detecção de clones por embeddings já existente.

Estado atual: foram traduzidas e verificadas manualmente 8 funções do BigCodeBench (dentro da meta de 5-10 do prototipo inicial), com 44/44 testes Python e 49/49 testes C# passando de forma real (compilação e execução efetivas, não apenas inspeção). Esse trabalho, porém, **ainda não foi commitado** no git. O núcleo do objetivo -- gerar variantes Type-IV em C# -- **ainda não foi implementado**: existe apenas tradução 1-para-1, não geração de clones diversificados. O lado Python do pipeline, por outro lado, é robusto e já tem resultados reais: 862 entradas e 6123 clones (média de 7.1 clones/entrada -- a meta de '~7 clones por entrada' já foi atingida, mas em Python).

Principais pendências: escalar o Golden Dataset de 8 para 100 entradas, implementar a geração de clones Type-IV em C# (prompt de diversificação + validação automatizada por compilação/execução), construir o dataset final de clones C#, e rodar a infraestrutura de embeddings sobre esse dataset (hoje ela já funciona com C#, mas só foi testada contra bases externas, nunca contra dados gerados pelo próprio Kamino).

3. Estado Atual do Projeto

Estimativa: **~20-25% do objetivo de extensão para C#**, calculada ponderando as 6 etapas do plano pelo peso relativo de cada uma e o quanto de cada uma está de fato pronto (detalhamento completo na seção correspondente do documento de análise técnica). As duas etapas mais pesadas do plano -- geração de clones Type-IV (30%) e avaliação com embeddings sobre o dataset C# nativo (15%) -- estão praticamente em zero.

Etapas do plano	Status
1. Estudo do pipeline Python + dataset	CONCLUIDA
2. Análise das diferenças Python vs. C#	CONCLUIDA
3. Protótipo de tradução (5-10 funções)	CONCLUIDA (não commitada)
4. Golden Dataset C# (100 entradas)	EM ANDAMENTO (8/100)
5. Geração de clones Type-IV C# (~7/entrada)	NAO INICIADA
6. Avaliação com embeddings sobre dataset Kamino-C#	NAO INICIADA (infra existe e funciona com C# externo)

4. O que ja foi desenvolvido

Estudo do Kamino existente:

Pipeline de 6 passos (Normalizacao, Geracao de Clones, Filtragem CodeBLEU, Testes, Reparo/Reprompt, Clustering) mapeado e documentado (doc/step1.md a doc/step6.md). Confirmado como funcional atraves dos resultados reais ja existentes no repositorio.

Traducao das 8 funcoes:

dataset/golden_dataset_csharp/ contem 8 entradas do BigCodeBench traduzidas manualmente/assistidas de Python para C#, cada uma com implementacao e testes originais preservados (python/task_func.py, python/test_task_func.py) e a traducao C# correspondente (csharp/TaskFunc.cs, csharp/TaskFuncTests.cs).

Testes e validacao:

Reexecutado agora mesmo (dataset/golden_dataset_csharp/_shared/verify_all.py): **44/44 testes Python passando e 49/49 testes C# passando**, com compilacao e execucao reais (nao apenas leitura de codigo). Todas as 8 entradas marcadas como EQUIVALENT.

Prompts / infraestrutura inicial:

pipeline/src/utils/prompts.py ganhou um contexto "csharp_translate" (traducao 1-para-1, nao geracao de variantes). pipeline/src/steps/translate_csharp.py reaproveita a chamada ao Ollama ja existente, mas nunca foi executado com sucesso -- nao ha servidor Ollama acessivel neste ambiente.

Infraestrutura de embeddings ja existente:

pipeline/src/clone_detection/detect.py ja aceita language="csharp" e ja tem resultados reais e comitados em results/RQ3/ e results/RQ4/ (ex.: modelo treinado no Kamino Python avaliado contra GPTCloneBench C# obtendo F1~0.96) -- mas somente contra datasets C# EXTERNOS, nunca contra um dataset C# gerado pelo proprio Kamino.

5. O que ainda precisa ser desenvolvido

- Escalar o Golden Dataset C# de 8 para 100 entradas do BigCodeBench
- Implementar geracao de clones Type-IV em C# (prompt de diversificacao -- nao existe hoje, so ha traducao 1-para-1)
- Gerar ~700 clones (100 entradas x ~7 clones), com diversidade sintatica e equivalencia semantica comprovadas
- Criar validacao automatizada de clones C# (compilar + rodar testes), hoje so existe um script isolado fora do pipeline principal
- Construir o dataset final de clones C# no formato compativel com o Kamino existente
- Rodar a infraestrutura de embeddings sobre esse novo dataset (hoje so foi testada com C# externo)
- Executar os experimentos de deteccao de clones e analisar os resultados
- Commitar todo o trabalho ja realizado (hoje esta todo em working tree, nao commitado)

6. Arquitetura / Pipeline Atual

Lado Python (funcional, com resultados reais):

BigCodeBench (HuggingFace) -> normalization.py -> dataset normalizado (927 entradas)
-> clone_gen.py (LLM via Ollama, 4 modelos x 2 estrategias x 4 contextos x 7 refatoracoes)
-> filtering.py (filtro CodeBLEU <= 0.4 + execucao real dos testes originais)
-> reprompt.py (reparo de clones quase-corretos, ate 2 tentativas com outro modelo)
-> filtering.py (filtro final: 100% dos testes precisam passar)
-> clustering.py (agrupamento por similaridade + selecao de representante por cluster)
-> dataset final: 862 entradas, 6123 clones (media 7.1/entrada)

Lado C# (parcial, desconectado do pipeline principal):

BigCodeBench (927 entradas disponiveis) -> selecao manual -> 8 entradas
-> traducao manual/assistida -> csharp/TaskFunc.cs + csharp/TaskFuncTests.cs
-> verify_all.py (script isolado: compila com csc.exe + roda testes)
-> 8 entradas verificadas EQUIVALENT (nao integrado ao pipeline oficial)
-> [GERACAO DE CLONES: NAO EXISTE]
-> [DATASET FINAL DE CLONES C#: NAO EXISTE]

7. Arquitetura / Pipeline Pretendido

BigCodeBench

-> Python (927 entradas normalizadas, ja existe)
-> C# Golden (traducao 1-para-1 verificada -- escalar de 8 para 100)
-> Validacao do Golden (compilar + rodar testes traduzidos -- ja funciona para 8)
-> Geracao Type-IV em C# (prompt de diversificacao + LLM -- NAO EXISTE, criar)
-> Validacao dos clones (compilar + rodar os MESMOS testes do Golden -- criar validate_with_csharp)
-> Filtro CodeBLEU (lang="c_sharp", ja suportado pela biblioteca codebleu, confirmado)
-> Clustering (reutiliza clustering.py sem mudanca)
-> Dataset final de clones C# (formato compativel com o Kamino existente)
-> Embeddings (finetune.py/detect.py, pequenos ajustes para reconhecer o dataset nativo)
-> Experimentos (precision/recall/F1/MCC, mesmo padrao ja usado)
-> Resultados e analise comparativa

8. O que ja pode ser reutilizado

Componente	Localizacao	Funcao	Reutilizacao
Comunicacao com LLM	clone_gen.py::call_ollama_chat()	Chamada HTTP generica ao Ollama	Direta, 100%
Clustering	clustering.py::run_clustering()	Agrupar clones por CodeBLEU, seleciona representante	Direta, 100% (agnostico de linguagem)
Calculo de eficiencia	utils/efficiency.py	Mede sobrevivencia por configuracao no funil	Direta, 100%
CodeBLEU	helper_functions.py::calc_completeness_codebleu()	Similaridade sintatica via pacote codebleu	Direta -- confirmado suporte a lang="c_sharp"
Extracao/pares para embeddings (C# externo)	helper_functions.py, GPT/SEMANAL_LANGUAGE_ADAPTERS["csharp"]	Extrai pares de codigo C# de datasets externos	Ja existe e ja e usada
Avaliacao de embeddings	clone_detection/detect.py::run_clone_evaluation()	Calcula precision/recall/F1/MCC	Ja aceita language="csharp"
Fine-tuning de embeddings	clone_detection/finetune.py::run_finetuning()	Treina SentenceTransformer sobre pares	Estrutura reutilizavel, so falta registrar dataset C# nativo
Extracao de codigo da resposta do LLM	translate_csharp.py::_extract_csharp_code()	Regex de bloco ```csharp	Ja criada, reutilizavel
Validacao Python (referencia de design)	helper_functions.py::validate_with_unittest()	Executa codigo + testes, devolve PASS/FAIL/ERROR	Nao reutilizavel diretamente (e Python-only); serve de modelo para criar o equivalente C#
Geracao de variantes (refatoracoes)	prompts.py::REFACTORING (refac_1..7)	Texto de instrucoes de diversificacao	Textos provavelmente reaproveitaveis quase como estao para C#

9. O que precisa ser implementado

Funcionalidade	Prioridade	Arquivos	Dependencias	Resultado esperado
Validacao C# integrada (compilar+rodar testes)	CRITICA	novo modulo (ref.: _shared/TestHarness.cs, verify_all.py)	Compilador C# decidido	validate_with_csharp() com mesmo contrato de validate_with_unittest()
Prompt de diversificacao C#	CRITICA	prompts.py	Nenhuma	Novo context_builder de geracao de variantes (nao traducao)
Loop de geracao de clones C#	CRITICA	clone_gen.py (generalizar) ou novo modulo	Ollama acessivel	Candidatos C# brutos por entrada
Escalar Golden Dataset 8->100	ALTA	dataset/golden_dataset_csharp/	Ambiente Python ok	100 entradas verificadas EQUIVALENT
Filtro CodeBLEU para C#	ALTA	filtering.py, helper_functions.py	tree-sitter-c-sharp instalado	Clones filtrados por diversidade sintatica
Export para formato Kamino nativo	ALTA	novo script	Clones C# gerados e validados	JSON no mesmo shape do dataset Python
Registrar dataset C# em finetune.py/detect.py	MEDIA	clone_detection/finetune.py, detect.py, config.py	Dataset final C# existente	Treino/avaliacao rodando sobre dados nativos C#
Commitar trabalho atual	ALTA	git	Nenhuma	Historico do git atualizado

10. Dependencias e Ambiente

Item	Status confirmado agora
Python	3.14 instalado, sem venv configurado no projeto
.NET / C#	Somente csc.exe legado (.NET Framework 4.8, C# 5). Sem SDK moderno instalado
Ollama	Nao instalado / nao acessivel (conexao recusada em localhost:11434)
GPU	Nenhuma GPU NVIDIA detectada (nvidia-smi ausente)
Dependencias pip (requirements.txt)	Nenhuma instalada (torch, sentence_transformers, pandas, datasets, sklearn, etc. ausentes)
tree-sitter-c-sharp	Nao esta no requirements.txt; existe no PyPI (confirmado, v0.23.5); necessario para CodeBLEU em C#
.env / HF_TOKEN	Arquivo .env nao existe neste ambiente

Observacao importante: o README do projeto descreve GPU como "opcional", mas o codigo de fine-tuning (clone_detection/finetune.py) encerra a execucao (sys.exit(1)) se CUDA nao estiver disponivel -- ou seja, o fine-tuning de embeddings nao roda sem GPU, na pratica.

11. Bloqueios

Bloqueio	Classificacao	Como resolver
Nenhum mecanismo de geracao de clones C# existe	CRITICO	Implementar prompt de diversificacao + loop de geracao
Sem validacao C# integrada ao pipeline	CRITICO	Criar validate_with_csharp() com mesmo contrato da versao Python
Ollama nao acessivel neste ambiente	CRITICO	Instalar Ollama + baixar modelo(s) em maquina com recursos
Sem GPU confirmada (fine-tuning exige CUDA)	CRITICO	Confirmar acesso a maquina com GPU antes da fase de embeddings
Ambiente sem dependencias/.venv/.env	IMPORTANTE	pip install -r requirements.txt + tree-sitter-c-sharp; criar .env
So csc.exe legado (C# 5) disponivel	IMPORTANTE	Decidir: instalar .NET SDK moderno ou aceitar a limitacao
Trabalho das 8 funcoes nao commitado	PENDENCIA	git add / git commit
finetune.py/detect.py nao reconhecem dataset Kamino-C# nativo	PENDENCIA	Pequenos ajustes pontuais nos dois arquivos

12. O que precisa ser fornecido/configurado (depende de decisao externa)

- Acesso a uma maquina com Ollama instalado e pelo menos 1 modelo baixado (local ou remoto)
- Decisao: instalar .NET SDK moderno ou seguir apenas com o compilador legado csc.exe
- Confirmacao de acesso a GPU (local ou remota) para a etapa de fine-tuning dos embeddings
- Decisao sobre quais modelos LLM usar para C# (os 4 do Python, ou um subconjunto menor)
- HF_TOKEN valido (.env), caso seja necessario re-rodar a normalizacao do BigCodeBench
- Decisao: reaproveitar as 8 entradas atuais como parte das 100, ou re-gerar tudo pelo processo automatizado
- Decisao sobre quantidade de configuracoes de prompt por entrada em C# (reduzir do total usado em Python, por causa do tempo de execucao)
- Confirmacao de quem tem acesso ao servidor Ollama remoto referenciado em ollama_config_remote.json (porta 3333)
- Decisao sobre o limiar de CodeBLEU para C# (usar 0.4 como no Python, ou recalibrar)

13. Plano de Execucao

FASE 1 - Preparacao do ambiente

Instalar dependencias (requirements.txt + tree-sitter-c-sharp), configurar .env, decidir e confirmar acesso a Ollama e GPU.

Arquivos envolvidos: requirements.txt, .env

FASE 2 - Consolidar as 8 funcoes existentes

Commitar todo o trabalho ja feito (golden_dataset_csharp/, prompts.py, docs).

Arquivos envolvidos: git add / commit

FASE 3 - Validacao C# integrada

Criar validate_with_csharp() com o mesmo contrato de validate_with_unittest(), testar contra as 8 entradas existentes.

Arquivos envolvidos: novo modulo de validacao

FASE 4 - Prompt de diversificacao + geracao piloto

Criar prompt de geracao de variantes C# e gerar os primeiros clones reais (nao traducao) para 1-2 entradas piloto.

Arquivos envolvidos: prompts.py, clone_gen.py

FASE 5 - Escalar Golden Dataset (8 -> 100)

Traduzir e verificar as 92 entradas restantes, seguindo o mesmo padrao das 8 atuais.

Arquivos envolvidos: dataset/golden_dataset_csharp/

FASE 6 - Geracao de clones em escala (~7/entrada)

Aplicar a geracao da Fase 4 as 100 entradas, com filtro de CodeBLEU e validacao automatizada.

Arquivos envolvidos: clone_gen.py, filtering.py

FASE 7 - Clustering e dataset final

Reduzir candidatos a representantes nao-redundantes (clustering.py, reutilizado sem mudanca) e exportar para o formato Kamino nativo.

Arquivos envolvidos: clustering.py, novo script de export

FASE 8 - Embeddings

Registrar o dataset C# nativo em finetune.py/detect.py e rodar treino + avaliacao.

Arquivos envolvidos: config.py, finetune.py, detect.py

FASE 9 - Avaliacao e analise dos resultados

Comparar resultados do dataset Kamino-C# nativo com os resultados ja existentes contra GPTCloneBench/SemanticCloneBench C# (externos).

Arquivos envolvidos: notebook novo ou secao em RQ3/RQ4.ipynb

FASE 10 - Documentacao final

Atualizar doc/step7, README, DOC.md com o estado final do trabalho.

Archivos involucrados: doc/step7_csharp_translation.md, README.md

14. Critérios de Conclusão

Golden Dataset:

- [~] 100 entradas C# (hoje: 8)
- [x] Cada entrada possui teste correspondente (valido para as 8 atuais)
- [x] Todas compilam (valido para as 8 atuais)
- [x] Todas passam nos testes (valido para as 8 atuais: 49/49)
- [x] Equivalência comportamental confirmada por execução real (valido para as 8 atuais)
- [] Trabalho commitado no git

Clones:

- [] ~7 clones por entrada em média (hoje: 0)
- [] Clones sintaticamente diferentes (CodeBLEU <= limiar definido)
- [] Comportamento equivalente (100% dos testes do Golden passando)
- [] Testes passando de forma automatizada (validate_with_csharp funcionando)
- [] Duplicatas removidas/agrupadas via clustering
- [] Dataset final armazenado no formato compatível com o Kamino

Avaliação:

- [] Dataset C# carregado por finetune.py/detect.py
- [] Modelo de embeddings treinado ou reaproveitado sobre o dataset C#
- [] Experimentos executados (múltiplos thresholds, como já é feito para Python)
- [] Métricas calculadas (precision/recall/F1/MCC)
- [] Resultados salvos em CSV
- [] Análise comparativa feita (C# Kamino nativo vs. C# externo)

15. Proximos Passos Imediatos

- Commitar o trabalho ja existente (8 entradas, prompts, documentacao)
- Confirmar acesso a Ollama (local ou remoto) e a pelo menos 1 modelo LLM baixado
- Confirmar acesso a GPU para a futura etapa de fine-tuning de embeddings
- Decidir entre instalar .NET SDK moderno ou manter o compilador legado csc.exe
- Instalar tree-sitter-c-sharp e testar `calc_codebleu(..., lang="c_sharp")` isoladamente
- Criar o modulo `validate_with_csharp()` e valida-lo contra as 8 entradas ja existentes
- Criar o prompt de diversificacao C# e gerar clones piloto para 1-2 entradas
- Definir critério de quantidade de configuracoes de prompt por entrada (reduzir do total usado em Python)

16. Conclusao

O projeto de extensao do Kamino para C# esta, hoje, na fase de prototipo de traducao: 8 de 100 entradas do Golden Dataset foram traduzidas e verificadas de verdade (compilacao e execucao reais), com 44/44 testes Python e 49/49 testes C# passando. Esse resultado e concreto e confiavel, mas ainda nao esta commitado no repositório. O nucleo do objetivo do projeto -- a geracao automatizada de clones Type-IV em C# (variantes sintaticamente diferentes, semanticamente equivalentes) -- ainda nao foi implementado; o que existe hoje e apenas uma traducao 1-para-1, nao um mecanismo de diversificacao.

A boa noticia e que boa parte da infraestrutura necessaria ja existe e e reaproveitavel: a comunicacao com o LLM (Ollama), o clustering, o calculo de similaridade sintatica (confirmamos que a biblioteca CodeBLEU ja suporta C#) e, principalmente, a infraestrutura de avaliacao por embeddings -- que ja roda com C# hoje, ainda que contra datasets externos. O trabalho que falta e concentrado em tres frentes: (1) criar a geracao de clones e a validacao automatizada especificas para C#, que hoje nao existem; (2) escalar o Golden Dataset de 8 para 100 entradas; e (3) conectar o novo dataset C# a infraestrutura de embeddings ja existente. Nenhuma dessas frentes exige reescrever o pipeline -- sao extensoes pontuais sobre uma base solida -- mas todas dependem de recursos que nao estao confirmados neste ambiente hoje (Ollama acessivel, GPU, e uma decisao sobre a versao do compilador C#).